ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 10

Դինամիկ անդամներ և լռելյայն կոնստրուկտոր

Աշխատանքի նպատակը: Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ փոփոխել դինամիկ զանգվածի չափը:

Տեսական մաս: Լռելյայն կոնստրուկտորը կանչվում է առանց արգումենտների: Դատարկ օբյեկտում n=0 և data=nullptr արժեքները նկարագրում են վավեր վիճակ: Չափը փոխելիս նախ հատկացնել նոր զանգված, փոխանցել տարրերը և միայն դրանից հետո ազատել հինը:

Աշխատանքի կատարման կարգը

1. Կարդալ ընդհանուր պահանջները և նշանակված անհատական տարբերակը:

2. Սահմանել դասի դաշտերը, դրանց սահմանափակումները և մեթոդները:

3. Իրականացնել պահանջվող գործողությունները և ստուգել սխալ մուտքերը:

4. Համեմատել ստուգիչ օրինակի արդյունքը ձեռքով ստացված հաշվարկի հետ:

Հարթության վեկտորներ

Տվյալներ: Ստեղծել VectorSet դաս՝ պահելով վեկտորների x բաղադրիչների զանգվածը և ընդհանուր y բաղադրիչը: Այստեղ p=y, վեկտորները (x[i],y) են:

Հաշվարկներ: Հաշվել վեկտորների երկարությունների գումարը՝ $L=\sqrt{(x^2+y^2)}$, գտնել ամենակարճ վեկտորի ինդեքսը և (1,1) վեկտորի հետ դրական սկալյար արտադրյալ ունեցող վեկտորների քանակը՝ x+y>0:

Թույլատրելի արժեքներ: x և y արժեքները վերջավոր իրական թվեր են: Իրականացում: Լռելյայն p=0: Դատարկ օբյեկտում append-ով ավելացնել օրինակի զանգվածի տարրերը՝ պահելով լռելյայն p-ն: Առանձին օբյեկտը ստեղծել օրինակի p-ով: Առաջինից

հեռացնել ինդեքս 1-ի տարրը և երկու օբյեկտների համար հաշվել արդյունքները:

Փորձնական տվյալներ: x={-4,-1,2,5}, y=2: Ինդեքսները սկսվում են 0-ից:

```
#include <iostream>
#include <cmath>
class VectorSet {
private:
    double* data;
    int n;
    double p;
public:
    VectorSet() : data(nullptr), n(0), p(0.0) {}
    VectorSet(const double* arr, int size, double param) {
        if (size < 0 || size > 30) {
            n = 0;
            data = nullptr;
            p = 0.0;
            return; }
        n = size;
        p = param;
        if (n > 0) {
            data = new double[n];
            for (int i = 0; i < n; ++i) {
                data[i] = arr[i]; }
        } else {
            data = nullptr;  } }
    ~VectorSet() {
        delete[] data;
    }
    VectorSet(const VectorSet&) = delete;
    VectorSet& operator=(const VectorSet&) = delete;
    bool append(double value) {
        if (n >= 30) return false;
        double* newData = new double[n + 1];
        for (int i = 0; i < n; ++i) {
            newData[i] = data[i];  }
        newData[n] = value;
        delete[] data;
        data = newData;
        n++;
        return true; }
    bool removeAt(int index) {
        if (index < 0 || index >= n || n <= 0) return false;
        if (n == 1) {
            delete[] data;
            data = nullptr;
            n = 0
```

```
         return true; }
        double* newData = new double[n - 1];
        for (int i = 0, j = 0; i < n; ++i) {
            if (i == index) continue;
            newData[j++] = data[i]; }
        delete[] data;
        data = newData;
        n--;
        return true; }
    void print() const {
        std::cout << "Size (n): " << n << ", Parameter (y/p): " << p << ", Data:
[";
        for (int i = 0; i < n; ++i) {
            std::cout << data[i] << (i == n - 1 ? "" : ", "); }
        std::cout << "]" << std::endl; }
    double calculateTotalLength() const {
        if (n == 0) return 0.0;
        double total = 0;
        for (int i = 0; i < n; ++i) {
            total += std::sqrt(data[i] * data[i] + p * p); }
        return total;  }
    int findShortestVectorIndex() const {
        if (n == 0) return -1;
        int bestIdx = 0;
        double minLen = std::sqrt(data[0] * data[0] + p * p);
        for (int i = 1; i < n; ++i) {
            double len = std::sqrt(data[i] * data[i] + p * p);
            if (len < minLen) {
                minLen = len;
                bestIdx = i; } }
        return bestIdx; }
    int countPositiveScalarProduct() const {
        if (n == 0) return 0;
        int count = 0;
        for (int i = 0; i < n; ++i) {
            if (data[i] + p > 0) {
                count++; }  }
        return count;  } };
int main() {
    double sampleX[] = {-4, -1, 2, 5};
    int sampleSize = 4;
    double sampleY = 2.0;
    VectorSet objEmpty;
    for (int i = 0; i < sampleSize; ++i) {
        objEmpty.append(sampleX[i]); }

    return 0;
}
```